Beyond Datalog: Optimizing ResIR Expressions with E-Graphs

Compiler Theory & Practice (CPS2000) | Task 6 – Short paper on e-graphs (10%)

Kurt Abela (150106L) | kurt.abela.24@um.edu.mt

Introduction **1**
1. The Datalog Baseline: Analysis and Limitations **2**
The Strengths of Task 5 2
The Optimization Bottleneck 2
2. E-Graphs and Equality Saturation **2**
The Mechanism of egg 2
3. Enhancing ResIR Math Expressions **2**
E-Graphs vs. Task 5 3
Conclusion **3**
Bibliography **4**

Introduction

The **COMPILER-CPS2000** project is a multi-stage compiler pipeline designed to process **ResIR**, a domain-specific intermediate representation. In Task 5, we implemented a robust **Live Variable Analysis** using **Soufflé Datalog**, which successfully identifies variables that are "live" across basic blocks and detects dead assignments. While Datalog excels at fixed-point dataflow analysis, it encounters significant limitations when applied to complex mathematical optimizations. This paper explores how **E-graphs** and the **equality saturation** technique, exemplified by the **egg** library, can enhance the optimization of ResIR math expressions beyond the capabilities of our current Datalog implementation.

1. The Datalog Baseline: Analysis and Limitations

Our current implementation uses a **Fact Extractor** (`analysis.cpp`) to translate the Control Flow Graph (CFG) into relational tables: `edge.facts`, `def.facts`, and `use.facts`.

The Strengths of Task 5

The Datalog rules in `analysis.dl` define the logical relationships of liveness:

- `live_out(B, X) :- edge(B, S),`
- `live_in(S, X).live_in(B, X) :- use(B, X).`

This approach is highly effective for global analysis, such as determining that `%zero` in `basic_math.resir` is a dead definition because its value does not propagate beyond `bb0`.

The Optimization Bottleneck

When attempting to optimize mathematical expressions, Datalog typically follows a "destructive" or "one-way" rewrite path. For instance, if we defined a rule to simplify `%x = bin sub %a, %a` to `%x = const 0`, the original form is lost. This leads to the **Phase Ordering Problem**: if we optimize $A \rightarrow B$, we might miss a later optimization that requires the structure of A. In complex ResIR expressions, the order in which rules are applied significantly dictates the quality of the final code.

2. E-Graphs and Equality Saturation

An **E-graph** (equivalence graph) is a data structure that represents a set of equivalent programs compactly. Unlike our Datalog relations that track variable locations, an E-graph tracks **Equivalence Classes (e-classes)** of expressions.

The Mechanism of `egg`

The `egg` library utilizes **Equality Saturation**. Instead of replacing expression A with expression B, it adds B to the same e-class as A.

- **Non-destructive:** It stores all versions of an expression simultaneously.
- **Saturation:** It applies all possible rewrite rules until no more information can be added to the graph.
- **Extraction:** After saturation, a cost function (e.g., minimum number of operations) selects the most efficient version from each e-class.

3. Enhancing ResIR Math Expressions

Consider the **abs_diff** function in **basic_math.resir**. Our current analysis identifies that **%d** is used in both **bb1** and **bb2**. However, if **abs_diff** contained complex algebraic identities, an E-graph could optimize it far better than Datalog.

E-Graphs vs. Task 5

1. **Complexity Handling:** In Task 5, we manually extract uses and defs. In an E-graph, the library would automatically recognize that **%is_neg = bin lt %d, %zero** is equivalent to other logical representations, allowing the compiler to choose the most efficient branch condition.
2. **Global Identities:** E-graphs can easily handle rules like associativity and commutativity without getting stuck in infinite loops—a common danger in recursive Datalog rules.
3. **Better ResIR Lowering:** During Task 4 (C code generation), we translate ResIR opcodes directly. If we integrated **egg** before this stage, we could ensure that the C code generated from **codegen.cpp** utilizes the most mathematically simplified version of every expression.

Conclusion

While the **Soufflé Datalog** approach in Task 5 provides a robust framework for flow-sensitive analysis like liveness and dead-code detection, it is not optimized for the vast, non-linear search space of mathematical simplification. By adopting **E-graphs** and **Equality Saturation**, the **COMPILER-CPS2000** could transition from a basic analyzer to a sophisticated optimizer, ensuring that ResIR math expressions reach their most efficient form without the risks of phase ordering or destructive rewrites.

Bibliography

1. **Willsey, M., et al.** (2021). *egg: Fast and Extensible Equality Saturation.* POPL. (The foundational paper for the **egg** library and modern E-graph implementation).
2. **Jordan, H., et al.** (2016). *Soufflé: On Logic Clouds.* (Documentation on the Datalog engine used in Task 5 for fixed-point analysis).
3. **Tate, R., et al.** (2009). *Equality Saturation: A New Approach to Optimization.* (The original theory behind using E-graphs for compiler optimizations).
4. **ResIR Project Specification.** (2026). Internal Documentation for COMPILER-CPS2000.